Лабораторная работа 65
Блочная сортировка Bucket sort
Сравнительный отчёт

Содержание

Постановка и контракт

Алгоритм и сложность

Улучшения и проверки

Производительность и точность

Интеграция ACOM и визуализация

Исследование промптов и выводы

Воспроизводимость и источники

Автор Фирсова Валерия Дмитриевна

Группа 24ТИДИ

Сравниваются решения одной задачи при кратком и подробном промпте. Таблица содержит медианы всех запусков. Работоспособность означает только прохождение заданных проверок.

Внимание: Параметры запусков отличаются: различие результата нельзя приписать только промпту.

Постановка задачи

Лабораторная 65 относится к подразделу 5.3 задания. Требуется блочная распределительная сортировка в новом компоненте ACOM и анализ скорости относительно стандартного аналога C. Разработан фиксированный интерфейс IEcoLab65 с методами SortInt, SortLong, SortFloat, SortDouble и SortLongDouble. Сортировка выполняется по возрастанию. int32_t и int64_t принимают весь знаковый диапазон; float, double и long double принимают конечные числа от -1000000 до 1000000 включительно. Максимальная длина 1000000, число блоков от 1 до 1024. Контракт различает успех 0, ошибку входа -1 и отказ аллокатора -2. При ошибке исходный массив должен оставаться неизменным. N=6 десятичных знаков задан до эксперимента; независимые проверки дополнительно требуют точного порядка и сохранения значений, включая количество отрицательных нулей.

Алгоритм

Обе ветки реализуют Bucket sort с сортировкой вставками внутри блоков. Численные методы используют общий макрос тела и пять буквальных ABI функций, различая исходный тип массива и функцию определения блока. Для знаковых целых значение переводится в беззнаковый тип той же ширины и XOR с маской старшего бита делает ключ монотонным. При B>1 ширина равна UMAX/B+1; при B=1 индекс равен нулю без переполнения ширины. Целочисленные значения сохраняются и сравниваются в исходном типе, без перехода через double. Вещественный индекс строится нормированием фиксированной области [-1000000,1000000], с отсечением верхнего индекса к B-1. В двух формах исходников порядок умножения и деления различается; это может менять пограничное распределение при округлении, но не округляет сами сохраняемые значения. До выделений проверяется весь вещественный массив; NaN/Infinity/недопустимые значения отвергаются.

Затем выделяются типизированный временный массив и три массива счётчиков: populations, starts и cursors. После обнуления populations выполняются подсчёт размеров и префиксные начала блоков. В кратких реализациях пар 1 и 3 и в подробных реализациях scatter заполняет временный массив, вставки сортируют его диапазоны, затем результат копируется в data. В краткой реализации пары 2 temp сохраняет исходную копию: scatter переносит её значения в data, а вставки сортируют диапазоны уже в data. Оба потока сохраняют исходные значения и выполняют записи только после полной валидации и успешных выделений. Поскольку отображение монотонно, конкатенация отсортированных блоков даёт общий порядок. При отказе выделения возвращается -2, все успешные выделения освобождаются, исходный массив не меняется.

В кратком исходнике пары 1 внутреннее перемещение уменьшает и внешний индекс j: это вызывает повторный просмотр уже упорядоченного префикса. Подробные исходники используют отдельный индекс для перемещения, сохраняя внешний scan. Краткие исходники пар 2 и 3 также разделяют внешний и внутренний индексы; одинаковый результат на тестах

сам по себе не означает одинакового количества операций. С qsort применяется только в независимом клиенте как стандартный аналог и эталон порядка с компараторами (a>b)-(a<b), без переполняющего вычитания.

Оценка сложности

Пусть n — длина, B — число блоков, n_i — размер i-го блока. Подсчёт, scatter, копирование и проверки требуют O(n+B); вставки дают O(sum(n_i^2)), поэтому общее время O(n+B+sum(n_i^2)). Лучший случай уже упорядоченных/одинаковых значений — O(n+B). При примерно равномерном распределении элементов между B блоками ожидаемое время O(n+B+n^2/B). При концентрации в одном блоке или B=1 худший случай O(n^2+B). Повторные просмотры краткого исходника пары 1 повышают число операций, сохраняя квадратичную верхнюю оценку. При фиксированном B=128 нельзя объявлять сортировку линейной для растущего n; контракт ограничивает B сверху 1024. Полный фиксированный диапазон отображения означает, что даже равномерный узкий интервал значений может попадать в один или несколько блоков; особенно это касается int64 с полным знаковым диапазоном. Быстрый путь для таких целых входов отдельно не измерялся. Дополнительная память — n*sizeof(T)+3*B*sizeof(uint32_t)+O(1), то есть O(n+B). Для double при n=1000000 и B=1024 основные буферы занимают 8012288 байт. Это память алгоритма, а не память модели MLX. Стандарт интерфейса qsort не задаёт конкретную реализацию или её асимптотику; устройство данной Darwin qsort отдельно не исследовалось, поэтому её сравнение здесь основано на замерах, а не обещании времени.

Сравнение работы ИИ

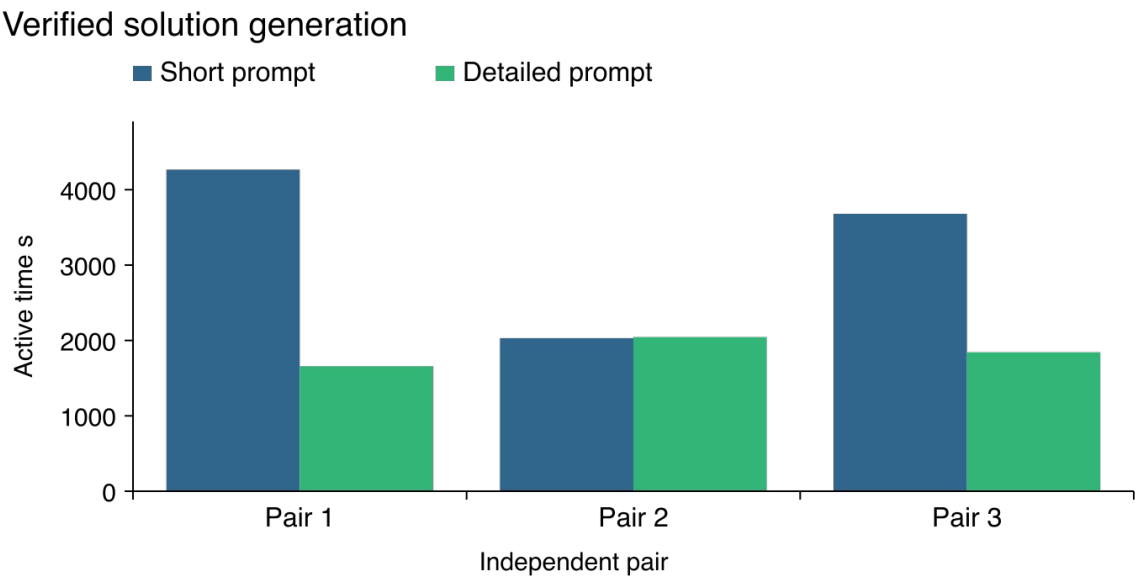
Показатель	short	detailed
Успешные / всего	3 / 3	3 / 3
Размер промпта, символы	377	3632
Ручные уточнения организатора	7	4
Активное время, с	3680.57	1843.96
Календарное время с паузами, с	19150.18	18065.19
Время API, с	3651.27	1830.42
Время проверок, с	15.96	12.86
Запросы HTTP	23	12
Входные токены	283673	141033
Выходные токены	36349	17956
Всего токенов	322823	158989
Циклы исправления	21	11

Входные токены суммируются по всем запросам вместе с повторно переданной историей и системным промптом. Ошибки сети учитываются как попытки HTTP; их токены могут быть неизвестны. Активное время включает API, инструменты, проверки и повторы, но исключает паузы между сессиями. Время выполнения лабораторного алгоритма измеряется отдельно.

Все запуски

Пара	Промпт	Активное время, с	Входные токены	Выходные токены	HTTP	Уточнения
1	Краткий	4266.21	283673	43827	23	10
1	Подробный	1659.33	126825	16682	11	4
2	Подробный	2045.15	160313	19649	13	4
2	Краткий	2030.58	172732	17730	15	5
3	Краткий	3680.57	286474	36349	26	7
3	Подробный	1843.96	141033	17956	12	4

Активное время получения проверенного решения



Улучшения

Исправления фактически включили полные пять методов, C89 размещение объявлений, буквальные ABI имена вместо ошибочного ##, сохранение знаковых/вещественных типов, монотонные ключи вместо остатка/усечения, защиту B=1, отсечение верхнего endpoint, инициализацию счётчиков, полную валидацию до Alloc и корректный Free(allocator,pointer). Volatile доступ к populations предотвращает автоматическое превращение обнуления в запрещённый libc bzero. Общий макрос уменьшает повторение пяти методов, но затрудняет диагностику и требует аккуратного разделения типов и имён. Некоторые исходники содержат повторную проверку области в mapper и лишние объявления индексов: прохождение тестов не означает оптимальность кода. Возможные дальнейшие улучшения — выбор блоков по данным и замена вставок для крупных переполненных блоков. Они в измеренные реализации не добавлены.

Результаты тестирования алгоритма

До генерации незавершённый шаблон компилировался в архив, но не проходил независимую проверку из-за отсутствия пяти численных методов. Все варианты проверяются одним замороженным набором: реальная ACOM регистрация, QueryInterface/AddRef/Release, ANSI C89, зависимости, UBSan, пять типов; длины 0,1,2,17,257,4096; B=1,2,7,128,1024; случайные,

отсортированные, обратные, одинаковые значения и повторы. Отдельно проверены знаковые экстремумы, соседние int64 за 2^53, конечные границы, малые вещественные значения, количество отрицательных нулей, null, недопустимые размеры/блоки, NaN/Infinity в хвосте и singleton, неизменность входа при ошибке, отказы каждого последовательного Alloc и освобождение успешных выделений. Легальный максимум 1000000 проверен для каждого типа на одинаковых значениях. Ожидаемые значения получены независимым C qsort с компараторами без вычитания. Успешные проверки требуют точного сохранения типизированных значений; округление до N=6 используется только для представления таблицы. short, пара 1: 1648 проверок, ошибок=0, живых выделений=0, неверных Free=0. detailed, пара 1: 1648 проверок, ошибок=0, живых выделений=0, неверных Free=0. detailed, пара 2: 1648 проверок, ошибок=0, живых выделений=0, неверных Free=0. short, пара 2: 1648 проверок, ошибок=0, живых выделений=0, неверных Free=0. short, пара 3: 1648 проверок, ошибок=0, живых выделений=0, неверных Free=0. detailed, пара 3: 1648 проверок, ошибок=0, живых выделений=0, неверных Free=0.

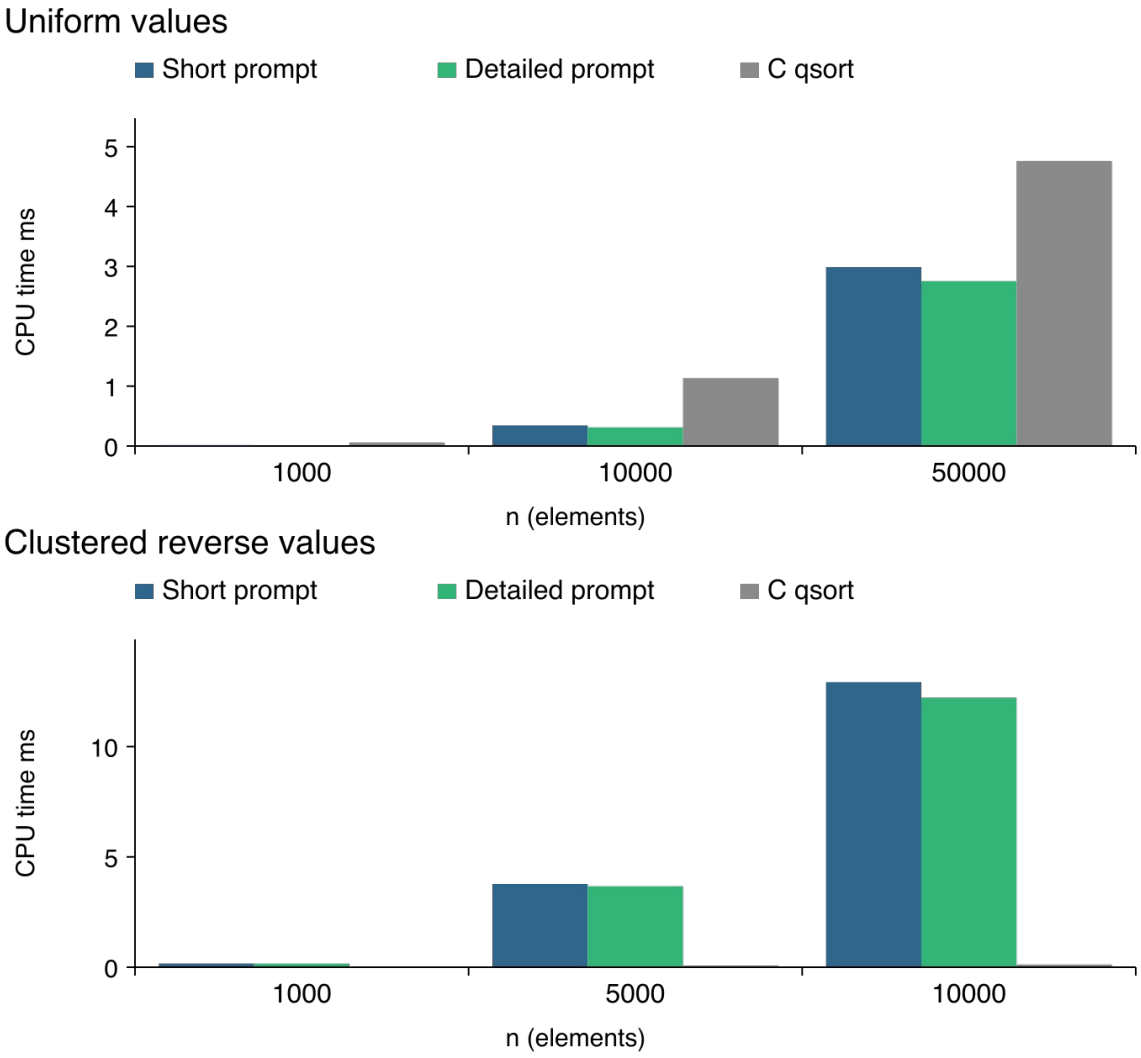
Пример SortDouble, шесть значений. Абсолютная ошибка относительно qsort:

Индекс	Получено	qsort	Абсолютная ошибка
0	-1000000.000000	-1000000.000000	0
1	-1.250000	-1.250000	0
2	0.000000	0.000000	0
3	0.500000	0.500000	0
4	0.500000	0.500000	0
5	1000000.000000	1000000.000000	0

Реальная скорость измерена O2 сборкой без UBSan, clock() (CPU секунды): 5 повторов исходников каждого из трёх запусков ветки, одинаковые данные, seed=1234567, B=128. Равномерные входы n=1000,10000,50000 и концентрированные в [100,101] обратные n=1000,5000,10000 зафиксированы до генерации. Вызов ACOM и аллокации включены, подготовка и точная проверка результата исключены. Очерёдность qsort/Bucket чередуется. Таблица показывает медиану 15 замеров на строку; серая серия qsort на графике — медиана всех 30 соответствующих замеров. Значение qsort/Bucket больше 1 означает меньшее время Bucket. Все 180 результатов сверены с qsort; сырые CPU секунды сохранены в benchmarks-raw.csv, полные журналы в benchmark.log каждой попытки.

Вариант	Распределение	n	B	Bucket мс	qsort мс	qsort / Bucket
short	uniform	1000	128	0.016000	0.057000	3.562
short	uniform	10000	128	0.345000	1.140000	3.304
short	uniform	50000	128	2.989000	4.747000	1.588
short	cluster_reverse	1000	128	0.170000	0.016000	0.094
short	cluster_reverse	5000	128	3.773000	0.070000	0.019
short	cluster_reverse	10000	128	12.921000	0.124000	0.010
detailed	uniform	1000	128	0.014000	0.052000	3.714
detailed	uniform	10000	128	0.312000	1.128000	3.615
detailed	uniform	50000	128	2.755000	4.789000	1.738
detailed	cluster_reverse	1000	128	0.167000	0.017000	0.102
detailed	cluster_reverse	5000	128	3.671000	0.070000	0.019
detailed	cluster_reverse	10000	128	12.220000	0.129000	0.011

Время сортировки по независимым измерениям



Поддержка компонентов ACOM

Используется Eco.Framework_20260918.zip с SHA256 2f23fffb9e81d4e5423153e2ff7236578ae5e947501a1e5c4168d3dc2b3c08b3; eco-wizard 0.2.2 для macOS arm64. IID IEcoLab65 D565835C-B1F8-40BC-8023-47E6EBAB0281, CID EcoLab65 AE784DFA-9C5E-49EC-A39F-168BF4120672 созданы wizard. Консольный клиент входит через EcoMain; main поставляет Eco.System1. Фабрика регистрируется через IEcoInterfaceBus1 и экземпляр запрашивается QueryComponent. Общие QueryInterface, AddRef, Release, фабрика и доступ к IEcoMemoryAllocator1 поставлены в seed и защищены от изменений. Сигнатуры имеют ECOCALLMETHOD и реальные типы SDK. Qwen генерирует только BucketMethods.c; измерения не охватывают написание фабрики и клиента с нуля. Компонент собран статически по ANSI C89; расширения SDK изолированы через -isystem. Стандартные библиотеки применяются только в независимых клиентах. Дополнительные Calculator-компоненты старых примеров, агрегирование и connection points для этой задачи не добавлялись.

Визуализация

В разделах сравнения приведены графики из реальных CPU замеров и активного времени получения решения по трём парам. Оси имеют единицы миллисекунд и секунд. Callback визуализация процесса сортировки не требовалась и не реализована.

Выводы

Отдельный прогрев: HTTP=1, вход=32, выход=2, время=2.30 с; записан в warmup.jsonl и исключён из парных метрик.

Пара 1, short, 20261007-161358-173d8373: 21 неуспешных проверок, 10 ручных уточнений, 10 сокращений контекста; HTTP=23, вход=283673, выход=43827, активное время=4266.21 с, итог passed.

Пара 1, detailed, 20261007-162321-15f16862: 10 неуспешных проверок, 4 ручных уточнений, 4 сокращений контекста; HTTP=11, вход=126825, выход=16682, активное время=1659.33 с, итог passed.

Пара 2, detailed, 20261007-162900-c05fd23f: 12 неуспешных проверок, 4 ручных уточнений, 4 сокращений контекста; HTTP=13, вход=160313, выход=19649, активное время=2045.15 с, итог passed.

Пара 2, short, 20261007-163738-338f2e28: 14 неуспешных проверок, 5 ручных уточнений, 5 сокращений контекста; HTTP=15, вход=172732, выход=17730, активное время=2030.58 с, итог passed.

Пара 3, short, 20261007-164351-bd72add2: 25 неуспешных проверок, 7 ручных уточнений, 7 сокращений контекста; HTTP=26, вход=286474, выход=36349, активное время=3680.57 с, итог passed.

Пара 3, detailed, 20261007-164952-3d045c8c: 11 неуспешных проверок, 4 ручных уточнений, 4 сокращений контекста; HTTP=12, вход=141033, выход=17956, активное время=1843.96 с, итог passed.

Ручной анализ первой краткой попытки после 20-го ответа выявил compiler-generated _bzero, который не отвергал замороженный автоматический фильтр nm. На этой же попытке продолжена генерация без изменения фильтра или флагов. Замечание, исходный хеш и метрики сохранены в organizer/manual-dependency-review.json. Итоговая проверка всех шести объектов методов разрешает только внешнюю зависимость _lab_allocator; факт проверен в final-audit.json.

Все шесть сохранённых попыток в трёх парах доведены до одинаковых независимых проверок. Медиана активного времени краткой ветки 3680.57 с, подробной 1843.96 с; HTTP медианы 23 и 12; токены 322823 и 158989. Ни одна исходная попытка не прошла проверки без продолжений. Эти числа описывают весь сохранённый путь исправлений. При адаптивных ручных уточнениях, смене KV-кеша и сокращениях истории нельзя доказать причинное преимущество только длины исходного промпта. Подробный текст задаёт больше требований, но модель всё равно нуждалась в конкретной диагностике компилятора и граничных случаев. Корректность и скорость подтверждены только на описанном наборе входов; точный результат qsort не снимает худшую квадратичную оценку при концентрации значений.

Равномерный вход, n=50000, B=128: Bucket краткой ветки 2.989000 мс, подробной 2.755000 мс; qsort соответственно 4.747000 и 4.789000 мс (медианы 15 замеров). Концентрированный обратный вход, n=10000, B=128: Bucket краткой ветки 12.921000 мс, подробной 12.220000 мс; qsort соответственно 0.124000 и 0.129000 мс (медианы 15 замеров). На измеренных равномерных входах Bucket быстрее данного C аналога; при концентрации в одном блоке обе ветки существенно медленнее qsort. Это соответствует росту затрат вставок и не является универсальным превосходством одной сортировки. Близость времени двух веток не доказывает равенства их производительности на иных входах.

Ограничения: три пары на одном Apple M4 Pro с 24 GiB RAM и фиксированным seed=42; повторы независимы по истории и workspace, но не являются случайной выборкой моделей, машин или seed. Организатор использовал диагностические знания предыдущих неудач при составлении следующих уточнений; готовые исходники между ветками не передавались. Продолжения поэтому не являются контролируемым сравнением только двух исходных текстов.

HTTP в таблице — запросы генерации к локальному MLX; служебные проверки готовности и скачивание весов в этот счётчик не входят, прогрев приведён отдельно. Ручные уточнения — сохранённые вызовы lab-resume организатором; автоматическая диагностика после каждого ответа учитывается в токенах и запросах, но не в счётчике ручных уточнений. После ручных уточнений сравнивается путь от исходного промпта до проверенного решения.

Сокращение рабочего контекста сохраняет архивы и затраты, его измеренное время добавлено к активному времени; оно меняет доступную модели историю.

Первоначальная серия выполнялась с 4-битным KV-кешем. После десяти запросов первой короткой попытки продолжения переведены на 8-битный KV-кеш при прежних пределах памяти и тех же 4-битных весах. Это общий дополнительный этап с сохранением всех неудач; смена кеша, истории и уточнений не позволяет выделить причинный эффект только длины исходного промпта. Профили записаны в организаторском журнале и в sessions/server_metadata.

Контракт ограничивает вещественную область и длину массива; за его пределами решение не проверено. Точность long double на Apple arm64 равна double (LDBL_MANT_DIG=53). C89, скрининг исходников, nm и UBSan не составляют формального доказательства корректности.

Производительность измерена только для double, B=128 и двух заданных распределений; CPU clock имеет конечное разрешение, тепловое состояние и фоновые задачи не фиксировались.

Подготовка окружения, скачивание, проектирование шаблона/проверок/промптов и ручное чтение кода не входят в активное время обёртки. Ожидание между продолжениями исключено; фабрика и клиент заранее даны обеим веткам. Локальная тарифная цена токенов 0 USD не измеряет энергию или стоимость ноутбука.

В журнале памяти максимум наблюдаемого active MLX 10.04 GiB, peak RSS 7.98 GiB. Установлены guideline MLX 13 GiB, wired 12 GiB, free cache 0, KV 4 bit в исходной серии и 8 bit в последующих продолжениях, порог остановки active MLX 13 GiB / RSS 16 GiB, опрос 0.5 с. Наблюдения дискретные и не доказывают отсутствия более коротких пиков.

Воспроизводимость

Использована mlx-community/Qwen2.5-Coder-14B-Instruct-4bit, 4-битные веса, ревизия 29efdbab55a161237ab1e432a3abaf6c7ae2b477. Компьютер Apple M4 Pro, 24 GiB RAM; Python 3.13.7, MLX 0.32.3 и mlx-lm 0.32.0. macOS 26.6.2. Обёртка проверена: 41 unittest и Ruff. Запросы выполнялись только через локальный MLX HTTP сервер. Нумерованные этапы и профили сервера сохранены для каждой сессии; команда с KV 4 bit ниже относится к начальной серии.

Модель /Users/valeriia_firs/Desktop/github/ARPO_solver_macos/models/qwen-14b; temperature=0.7; top_p=0.8; seed=42; max_tokens=8192. Протокол single-file. Дополнительные параметры API: {"repetition_penalty": 1.05, "top_k": 20}. Общий контекст шаблона: 7657 символов (входит во входные токены). Ревизия весов: 29efdbab55a161237ab1e432a3abaf6c7ae2b477. Eco SDK: Eco.Framework_20260918.zip; SHA256 архива: 2f23fffb9e81d4e5423153e2ff7236578ae5e947501a1e5c4168d3dc2b3c08b3. Сохранённая команда сервера: /Users/valeriia_firs/Desktop/github/ARPO_solver_macos/.venv/bin/python /Users/valeriia_firs/Desktop/github/ARPO_solver_macos/labs/bucket-sort-65/launch_limited.py --model /Users/valeriia_firs/Desktop/github/ARPO_solver_macos/models/qwen-14b --host 127.0.0.1 --port 8080 --prompt-cache-size 0 --prefill-step-size 128 --kv-bits 4 --quantized-kv-start 0 --decode-concurrency 1 --prompt-concurrency 1.

Каталог результатов: /Users/valeriia_firs/Desktop/github/ARPO_solver_macos/labs/bucket-sort-65/runs/experiment-20261007-161356-588ed87a. Подкаталоги соответствуют ID в истории попыток раздела «Выводы». Исходные промпты находятся в prompt.txt, уточнения в followup-*.txt.

- short: промпт SHA256 c0967ee83704fe098824d1c2d519777c7b19dfbf47251483ea1f3d5177a79037.

- detailed: промпт SHA256 bab4db8fbe9558ef4d821cf24835efabf38d50bf45d82957b348d8412fd0c741.

Полная конфигурация, версии окружения, запросы, ответы, проверки и исходники сохранены рядом с run.json. Стоимость по тарифам не включает электричество и амортизацию ноутбука. При нулевых тарифах локального сервера плата провайдеру равна нулю; объём вычислений отражают токены и время.

Источники

- Список заданий.pdf пользователя, страницы 1-3 и 6-8: ограничения ACOM, типы, тема 65, обязательность UnitTest, Word и замеров производительности.

- пример оформления 1.pdf и пременр оформления 2.pdf пользователя: образцы структуры и оформления отчёта; реализации других сортировок не включены в эксперимент.

- NIST Dictionary of Algorithms and Data Structures, Bucket sort: https://xlinux.nist.gov/dads/HTML/bucketsort.html

- The Open Group, qsort: https://pubs.opengroup.org/onlinepubs/009695399/functions/qsort.html

- Apple, Writing ARM64 code for Apple platforms: https://developer.apple.com/documentation/xcode/writing-arm64-code-for-apple-platforms

- MLX, set_memory_limit: https://ml-explore.github.io/mlx/build/html/python/_autosummary/mlx.core.set_memory_limit.html

- MLX model weights, pinned revision: https://huggingface.co/mlx-community/Qwen2.5-Coder-14B-Instruct-4bit/tree/29efdbab55a161237ab1e432a3abaf6c7ae2b477

- Локальные контракты, независимые проверки, исходные промпты, run.json, requests.jsonl, events.jsonl и benchmark.log данной лабораторной.

Исходные промпты и продолжения

Исходный промпт short
Implement Bucket sort for the five functions declared in HeaderFiles/ArpoMethods.h. Follow its contract exactly. Use ANSI C89, only the supplied Eco allocator for temporary memory, and no libc, math.h or third-party calls. The ACOM scaffold and client are supplied. Return one complete SourceFiles/BucketMethods.c including ArpoMethods.h; do not change the ABI or other files.

Уточнение 1 short, 20261007-161358-173d8373
Your previous file still repeats known compiler errors and leaves four methods unimplemented. Produce a complete working replacement, not another cosmetic edit. Review the original contract again. The fixed declarations… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-161358-173d8373/followup-01.txt.

Уточнение 2 short, 20261007-161358-173d8373
Write a NEW complete implementation from the contract, using actual ANSI C89 syntax. The previous unsuccessful code is archived and should not be reused. Provide exactly one fenced source file for BucketMethods.c. Use de… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-161358-173d8373/followup-02.txt.

Уточнение 3 short, 20261007-161358-173d8373
Reimplement the entire file with all five methods, using the precise specification below. Previous failed files are archived; do not copy their incorrect structure. The allocator helper is already supplied by the common… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-161358-173d8373/followup-03.txt.

Уточнение 4 short, 20261007-161358-173d8373
Transform YOUR LAST SOURCE FILE into strict ANSI C89. Do not return the same file. The compiler rejected it. Return the full corrected file only. 1. Remove the word inline from EVERY helper definition. C89 has no inline… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-161358-173d8373/followup-04.txt.

Уточнение 5 short, 20261007-161358-173d8373
Correct your last complete source. Most exported methods now use C89 declarations correctly; preserve that syntax. Return the whole corrected file in one code block. The four remaining compiler errors are SPECIFIC: - val… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-161358-173d8373/followup-05.txt.

Уточнение 6 short, 20261007-161358-173d8373
Produce a NEW compact complete file. The prior duplicated functions keep retaining forbidden syntax. Use ONE type-parameterized C89 macro for the common sorting body and five exact

exported wrappers. Pass ordinary identi… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-161358-173d8373/followup-06.txt.

Уточнение 7 short, 20261007-161358-173d8373
Fix your latest compact macro source. Return the full corrected file. The shared sorting macro must contain ONLY a block of function-body statements and declarations. It must NOT generate a function definition or constru… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-161358-173d8373/followup-07.txt.

Уточнение 8 short, 20261007-161358-173d8373
Your latest source now COMPILES in C89 and has all five ABI functions. Preserve that working syntax and structure. The independent client crashes because the three floating mappers return an out-of-bounds index for the L… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-161358-173d8373/followup-08.txt.

Уточнение 9 short, 20261007-161358-173d8373
Your latest file passes all 1648 automated cases. Manual object-symbol review nevertheless found _bzero in BuildFiles/Arpo/component-000.o (BucketMethods.c). The no-libc contract applies to this compiler-generated depend… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-161358-173d8373/followup-09.txt.

Уточнение 10 short, 20261007-161358-173d8373
The only compilation error in your latest file is the populations Free call. You mistakenly passed the buffer as the first argument and a byte count as the second. Read the supplied IEcoMemoryAllocator1 contract: Free ta… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-161358-173d8373/followup-10.txt.

Исходный промпт detailed
Implement Bucket sort for the five functions declared in HeaderFiles/ArpoMethods.h. Follow its contract exactly. Use ANSI C89, only the supplied Eco allocator for temporary memory, and no libc, math.h or third-party calls. The ACOM scaffold and client are supplied. Return one complete SourceFiles/BucketMethods.c including ArpoMethods.h; do not change the ABI or other files.

Use real distribution into buckets, followed by sorting within each bucket and concatenating buckets. Do not replace the entire algorithm with comparison sorting. Copy all five declarations literally; their data types are int32_t, int64_t, float, double, long double, with uint32_t count and buckets. You may use C89 macros to share structure while preserving each exact type.

Validate me, count <= 1000000, buckets from 1 through 1024, and data when count > 0. Even count=0 must reject invalid me/buckets/count; valid empty input succeeds without allocating. For each floating type first scan ALL elements and reject NaN (x != x), infinity and values outside [-1000000,1000000]. Validate singleton values too. Return -1 on invalid input without changing any element. Return -2 on allocation failure, release every successful allocation, and leave the original array unchanged. Return 0 on success. Never allocate through malloc/calloc/realloc/free.

Suggested algorithm: obtain IEcoMemoryAllocator1* with lab_allocator(me). Allocate a typed temporary array and bucket count/start/cursor arrays with Alloc, whose size argument is uint32_t. The fixed count limit makes byte sizes fit. Initialize counters with loops; count bucket populations, construct prefix offsets, scatter ORIGINAL typed values into temporary bucket regions, sort each region ascending with insertion sort, then copy sorted values back using typed assignments and free

all memory. Complete every allocation before modifying the original array. Treat no-op cases safely. No memcpy/memset/qsort, including compiler-emitted dependencies; explicit loops and scalar operations only.

Bucket index must be monotonic and bounded in [0,buckets-1]. Signed integers span their FULL domains, including minimum and maximum; do not subtract signed min/max or negate a signed minimum. Form an unsigned ordered key by converting to uint32_t/uint64_t and toggling the high sign bit. For buckets>1 use key divided by (((unsigned maximum)/buckets)+1); for buckets==1 index=0 to avoid width overflow. Construct 64-bit constants from SDK uint64_t shifts, not C99 ULL constants. Never cast integer data to double for sorting, since adjacent int64_t values above 2^53 must stay distinct.

For floating values map the fixed interval [-1000000,1000000] to the bucket interval using wider arithmetic as needed. Clamp the upper endpoint to buckets-1 after validation. Sort and copy using the original type; no quantization. Duplicates, sorted/reverse input, negative values, both endpoints, signed zero, very small values and highly clustered inputs must work. The accuracy target is exact equality to the independently sorted values, also reported to N=6 decimal places; sorting must preserve the original multiset.

Insertion sorting inside buckets has expected O(n+B+n^2/B) work for uniform independent keys and worst O(n^2+B) for a concentrated reverse sequence; auxiliary space O(n+B). Performance will be measured separately on uniform and clustered data against C qsort. Comments should describe the implementation and limitations. Do not redeclare ACOM interfaces, IID/CID, factory or lifetime methods. Do not access PDFs or tests outside the workspace. Output a complete compilable source file only.

Уточнение 1 detailed, 20261007-162321-15f16862
Reimplement the complete file from the ORIGINAL fixed contract. Previous unsuccessful source is archived; do not reuse its errors. Return exactly ONE full C89 source code block for BucketMethods.c, with all five methods… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-162321-15f16862/followup-01.txt.

Уточнение 2 detailed, 20261007-162321-15f16862
Transform YOUR LAST SOURCE FILE into strict ANSI C89. Do not return the same file. The compiler rejected it. Return the full corrected file only. 1. Remove the word inline from EVERY helper definition. C89 has no inline… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-162321-15f16862/followup-02.txt.

Уточнение 3 detailed, 20261007-162321-15f16862
Your current file now compiles in C89. Keep that declaration placement and all five literal ABI signatures. The independent tests fail because bucket order and typed values are corrupted. Return the complete corrected fi… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-162321-15f16862/followup-03.txt.

Уточнение 4 detailed, 20261007-162321-15f16862
Your latest typed-helper source has correct monotone unsigned integer mappings, clamped floating mappings, typed scratch storage, volatile populations, prefix/scatter passes, separate insertion indices and the correct Fr… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-162321-15f16862/followup-04.txt.

Уточнение 1 detailed, 20261007-162900-c05fd23f
Reimplement the complete file from the ORIGINAL fixed contract. Previous unsuccessful source is archived; do not reuse its errors. Return exactly ONE full C89 source code block for BucketMethods.c, with all five methods… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-162900-c05fd23f/followup-01.txt.

Уточнение 2 detailed, 20261007-162900-c05fd23f
Transform YOUR LAST SOURCE FILE into strict ANSI C89. Do not return the same file. The compiler rejected it. Return the full corrected file only. 1. Remove the word inline from EVERY helper definition. C89 has no inline… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-162900-c05fd23f/followup-02.txt.

Уточнение 3 detailed, 20261007-162900-c05fd23f
Your current file now compiles in C89. Keep that declaration placement and all five literal ABI signatures. The independent tests fail because bucket order and typed values are corrupted. Return the complete corrected fi… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-162900-c05fd23f/followup-03.txt.

Уточнение 4 detailed, 20261007-162900-c05fd23f
Your latest typed-helper source has correct monotone unsigned integer mappings, clamped floating mappings, typed scratch storage, volatile populations, prefix/scatter passes, separate insertion indices and the correct Fr… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-162900-c05fd23f/followup-04.txt.

Уточнение 1 short, 20261007-163738-338f2e28
Reimplement the complete file from the ORIGINAL fixed contract. Previous unsuccessful source is archived; do not reuse its errors. Return exactly ONE full C89 source code block for BucketMethods.c, with all five methods… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-163738-338f2e28/followup-01.txt.

Уточнение 2 short, 20261007-163738-338f2e28
Transform YOUR LAST SOURCE FILE into strict ANSI C89. Do not return the same file. The compiler rejected it. Return the full corrected file only. 1. Remove the word inline from EVERY helper definition. C89 has no inline… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-163738-338f2e28/followup-02.txt.

Уточнение 3 short, 20261007-163738-338f2e28
Your current file now compiles in C89. Keep that declaration placement and all five literal ABI signatures. The independent tests fail because bucket order and typed values are corrupted. Return the complete corrected fi… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-163738-338f2e28/followup-03.txt.

Уточнение 4 short, 20261007-163738-338f2e28
Your latest file compiles in C89 and has correct typed temporary storage and monotone/clamped helpers, but it ONLY DISTRIBUTES values into buckets. It never sorts WITHIN the buckets. That is why the demo still prints uns… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-163738-338f2e28/followup-04.txt.

Уточнение 5 short, 20261007-163738-338f2e28
Repair YOUR LATEST in-place bucket source. Keep its correct helpers, original typed values, immutable temp copy, population/prefix/cursor scatter into data, volatile zero stores, four allocations and cleanup. Return the… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-163738-338f2e28/followup-05.txt.

Уточнение 1 short, 20261007-164351-bd72add2
Reimplement the complete file from the ORIGINAL fixed contract. Previous unsuccessful source is archived; do not reuse its errors. Return exactly ONE full C89 source code block for BucketMethods.c, with all five methods… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-164351-bd72add2/followup-01.txt.

Уточнение 2 short, 20261007-164351-bd72add2
Transform YOUR LAST SOURCE FILE into strict ANSI C89. Do not return the same file. The compiler rejected it. Return the full corrected file only. 1. Remove the word inline from EVERY helper definition. C89 has no inline… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-164351-bd72add2/followup-02.txt.

Уточнение 3 short, 20261007-164351-bd72add2
Your current file now compiles in C89. Keep that declaration placement and all five literal ABI signatures. The independent tests fail because bucket order and typed values are corrupted. Return the complete corrected fi… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-164351-bd72add2/followup-03.txt.

Уточнение 4 short, 20261007-164351-bd72add2
Your latest C89 file corrupts values because it reads temp[i] while its insertion/scatter loop writes into the SAME temp array. Earlier writes destroy original values that have not yet been read. The demo duplicates valu… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-164351-bd72add2/followup-04.txt.

Уточнение 5 short, 20261007-164351-bd72add2
Your latest C89 file crashes: the scatter loop starts unsigned j at cursors[index]-1, which wraps to a huge index for an empty bucket beginning at0, and it reads data that has not yet been filled. DO NOT sort while scatt… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-164351-bd72add2/followup-05.txt.

Уточнение 6 short, 20261007-164351-bd72add2
Reconstruct the COMPLETE file from the original fixed contract, not from the repeatedly crashing partial source. Return ONE full C89 code block. All old responses remain archived. Do not copy its integrated scatter/inser… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-164351-bd72add2/followup-06.txt.

Уточнение 7 short, 20261007-164351-bd72add2
The latest reconstructed common body now has the correct parameter/floating validation order and separate temp scatter/sort/copy, but two concrete defects remain. Return the COMPLETE file, preserving the five literal wra… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-164351-bd72add2/followup-07.txt.

Уточнение 1 detailed, 20261007-164952-3d045c8c
Reimplement the complete file from the ORIGINAL fixed contract. Previous unsuccessful source is archived; do not reuse its errors. Return exactly ONE full C89 source code block for BucketMethods.c, with all five methods… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-164952-3d045c8c/followup-01.txt.

Уточнение 2 detailed, 20261007-164952-3d045c8c
Transform YOUR LAST SOURCE FILE into strict ANSI C89. Do not return the same file. The compiler rejected it. Return the full corrected file only. 1. Remove the word inline from EVERY helper definition. C89 has no inline… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-164952-3d045c8c/followup-02.txt.

Уточнение 3 detailed, 20261007-164952-3d045c8c
Your current file now compiles in C89. Keep that declaration placement and all five literal ABI signatures. The independent tests fail because bucket order and typed values are corrupted. Return the complete corrected fi… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-164952-3d045c8c/followup-03.txt.

Уточнение 4 detailed, 20261007-164952-3d045c8c
The last C89 file compiles and its typed scatter/insertion/mapping are now coherent. UBSan fails because the floating wrappers read data before checking their parameters. Make a narrow correction to ALL THREE wrappers la… Полный текст: runs/experiment-20261007-161356-588ed87a/20261007-164952-3d045c8c/followup-04.txt.

Контроль происхождения исходников

short, пара 1: BucketMethods.c SHA256
251d3699f63d8754de379eb60ff4af347ec369b3ee0da0580c6a5fdd75c546a0

detailed, пара 1: BucketMethods.c SHA256
93e1802eb73eceae642198f422985b0ff927e6355fe3b7a8749b38abfb4ce70c

detailed, пара 2: BucketMethods.c SHA256
a2e97ec431473dc9bd0d83eb87a942c18d4ff301b0871eb76a92e8fe24e3e34d

short, пара 2: BucketMethods.c SHA256
e6f51cc043cb38e077d0e6cca912f0cfcd6d877ad6033f9acee4f246ce9d9638

short, пара 3: BucketMethods.c SHA256
33e2ed06d773d7936dc01cadd930520520f4e454bbcbff9c7e820dd7fffc5ee1

detailed, пара 3: BucketMethods.c SHA256
d5a76ceec4fece2c4330a4b89bbf8a88b378294527073ceb4e27ec8de39e399c

Файлы сверены с последним ответом Qwen; суммы usage и хеши независимых проверок проверены отдельно в final-audit.json. Полные уточнения и архивы сокращённого контекста сохранены в каталогах запусков.